

Applied Analytics Project
March Madness Jupyter Notebook/Code Peer Review
Geovanni Jones, Joshua Previte and Casey Barrasso

The notebook is well organized and reflects a complete modeling pipeline: setup, ingestion, cleaning, feature engineering, modeling, evaluation, explainability, bias, deployment, monitoring, and inference. The first suggested improvement would be to make the project objective explicit near the top: for example, state that the goal is to predict whether team1 wins a tournament matchup **using pre-tournament team features and matchup differentials**.

SETUP

The setup section is strong. You import the necessary packages, define a project directory, create output folders, and set a random seed. This makes the notebook more reproducible.

DATA INGESTION

The data ingestion section is clear and simple. You load the main season data, team spelling map, and game statistics, then print their shapes. This is good practice because it confirms the files loaded correctly. One improvement would be to add a quick check of column names for each dataset (team_spellings and game_stats), especially since later steps depend heavily on exact column names like Mapped ESPN Team Name, Seed, and Post-Season Tournament. This would make the code easier to debug if the raw files change.

DATA CLEANING

This is one of the strongest sections of the notebook. The code maps team names to numeric team IDs, filters to March Madness teams, and builds a symmetric matchup structure where team1 is the lower TeamID and team2 is the higher TeamID. The leakage guard is especially important and well explained. Keeping only pre-tournament fields is a smart decision because it prevents the model from using information that would not be available before the game.

FEATURE ENGINEERING

Differential feature engineering is a good modeling choice. By creating features such as team1_stat - team2_stat, the model focuses on relative matchup strength rather than each team's raw characteristics. This is appropriate for predicting game outcomes because tournament results depend on the comparison between two teams.

The shooting-efficiency feature section is useful, but there is a possible leakage concern. The rates are calculated from game_stats grouped across the entire season. If game_stats includes tournament games, then these efficiency features may include information from the tournament itself, which would not be known before the matchup. Just to highlight a potential leakage issue.

EXPLORATORY DATA ANALYSIS

The EDA section is helpful and focused. The first chart identifies which differential features separate winners from losers, and the second chart shows how seed differential relates to team 1 win probability. These are relevant visuals because they connect directly to model intuition.

One improvement would be to add a brief interpretation after each chart. For example, after the seed differential chart, explain whether better-seeded teams win more often and whether the relationship looks linear or nonlinear. This would make the notebook stronger as a presentation artifact, not just a code file.

PREPROCESSING VALIDATION

Checking dataset shape, duplicate rows, class balance, and missing values is good practice. The class balance check is especially relevant because it affects model evaluation and whether accuracy is reliable.

TRAIN/VALIDATION/TEST SPLIT

The chronological split is appropriate because the model should be evaluated on future seasons, not randomly mixed games. Using seasons up to 2021 for training, 2022 for validation, and 2023–2025 for testing is a defensible approach. Also, your DROP_COLS list is good, but you should double-check whether any post-game box score features remain in the feature set. The line `select_dtypes(include=[np.number])` is convenient, but it can accidentally keep numeric leakage columns if they were not explicitly dropped.

BASELINE MODEL - LOGISTIC REGRESSION

The logistic regression pipeline is well built. It includes imputation, scaling, and a regularized logistic regression model. This is a good baseline because it is interpretable and provides a comparison point for the more complex Random Forest model.

One improvement would be to tune the logistic regression model using validation data or cross-validation. You currently use `C=1.0` by default. Testing several `C` values could show whether regularization improves generalization. Also, because your target may be somewhat balanced but not perfectly balanced, you could compare `class_weight=None` versus `class_weight='balanced'` if it was not already tested.

FINAL MODEL - RANDOM FOREST

The Random Forest model is a reasonable final model because it can capture nonlinear relationships and interactions between features such as seed differential, efficiency differential, and pre-tournament ratings. The pipeline correctly includes imputation, and the model uses `random_state` and `n_jobs=-1`, which is good.

However, the Random Forest appears mostly untuned. Parameters such as `max_depth=None`, `min_samples_split=2`, and `n_estimators=200` may lead to overfitting. I recommend tuning `max_depth`, `min_samples_leaf`, `min_samples_split`, and `max_features`. For example, testing shallower trees could improve test performance and reduce variance, if not already explored..

MODEL EVALUATION

The evaluation section is clear. Comparing logistic regression and Random Forest across train, validation, and test splits. Including accuracy, precision, recall, F1, and ROC-AUC gives a well-rounded view of model performance. Also, the confusion matrix is helpful, but it would be stronger if paired with a short explanation: for example, whether the model is better at predicting team1 wins or losses.

MODEL EXPLAINABILITY

The explainability section is useful because it includes both built-in Random Forest importance and permutation importance. Built-in importance can be biased toward continuous or high-cardinality features, so using permutation importance on the test set is a strong addition. However, the permutation importance is calculated on a surrogate model trained only on the top built-in features. That is acceptable for speed, but it should be clearly described as a surrogate explanation rather than a full explanation of the deployed model. If runtime allows, run permutation importance directly on the final pipeline using all features or at least explain why the surrogate was necessary.

BIAS DETECTION AND MITIGATION

This is a thoughtful section. Since the dataset does not contain protected demographic categories, using seed-difference buckets such as chalk, tossup, and upset as proxy groups is a reasonable fairness-style audit. It helps evaluate whether the model behaves differently across matchup types. The mitigation step uses `class_weight='balanced'`, but this addresses class imbalance rather than necessarily seed-group bias. If the bias concern is that the model overpredicts favorites, you might also compare accuracy, recall, and predicted positive rate across the seed buckets before and after mitigation. You could also test a model without seed features to see how much the model depends on seed differential.

MODEL DEPLOYMENT

The deployment section is practical and well done. Saving a dictionary containing the model, feature columns, threshold, version, and model family is better than saving only the raw model because it preserves important deployment metadata.

Also, include the Python version and package versions in a `requirements.txt` file for reproducibility. The following code could do this - `!pip freeze > requirements.txt`

MONITORING - DATA DRIFT & CONCEPT DRIFT

The monitoring section is strong and shows production thinking. PSI is a good metric for tracking whether feature distributions shift over time. Comparing baseline and recent accuracy also helps identify possible concept drift. One issue is that the same trained model is evaluated on historical baseline and recent slices of the full matchup dataset. This is useful as a demonstration, but in production you would monitor only newly arriving data and outcomes after games are played. The monitoring plan is clear, but it would be stronger if you defined what happens after an alert, such as retraining, feature audit, or rollback.

PRODUCTION INTERFACE EXAMPLE

The inference example is a good final demonstration. It loads the saved model, selects three held-out matchups, generates win probabilities, predicts the winner, and compares against the actual winner. This makes the project feel deployable.